Classifying Satellite Imagery with CNN Models

Jacob M. Knodle
North Dakota State University
Department of Statistics — Department of Computer Science

May 2026

Data Usage Notice
Data used under Creative Commons license for educational research purposes.
Full source information provided in citations.

1 Abstract

The goal of this project is to develop a deep learning image classification system that is specifically optimized for classifying different types of land-use and land-cover from aerial satellite images. A Convoluted Neural Network (CNN) was developed with the use of TensorFlow and Keras. The model was trained using a standard 80:20 training/validation split with the primary objective of the project is to demonstrate how Convoluted Neural Networks can be leveraged to automatically detect different image classes, specifically in the context of land classification tasks.

2 Introduction

Developments in deep learning networks and growing availability to satellite imagery via GIS has made it possible to construct automated satellite image classification models.

2.1 Data

The dataset utilized for the training and validation of the classification model contains 27,000 images obtained through GIS. These images are divided into ten separate classes, each with 2,000-3,000 images. These classes are defined by the nature of the land cover and major landmarks:

Land Cover Class	Number of Sample Images
Annual Crop Cover	3,000
Forest Cover	3,000
Herbaceous Vegetation Cover	3,000
Highway	2,500
Industrial District	2,500
Pasture Land	2,000
Permanent Crop Cover	2,500
Residential District	3,000
River	2,500
Sea/Lake	3,000
Total	27,000

Table 1: Land cover subset class size

2.2 Image Reformatting

Images were stored in RGB format via .jpg files before being resized to 64x64 pixels and normalized.

2.3 Dataset Splitting

A stratified train-validation split was utilized to avoid class imbalances across both the training and testing sets.

Stratification maintained proportional representation across the training set to ensure good model fit and the validation set to ensure accurate accuracy measurements.

Parameter	Value
Validation Split	20%
Batch Size	32
Epochs	25
Learning Rate	0.001
Random Seed	42

Table 2: Training Configuration

2.4 Data Augmentation

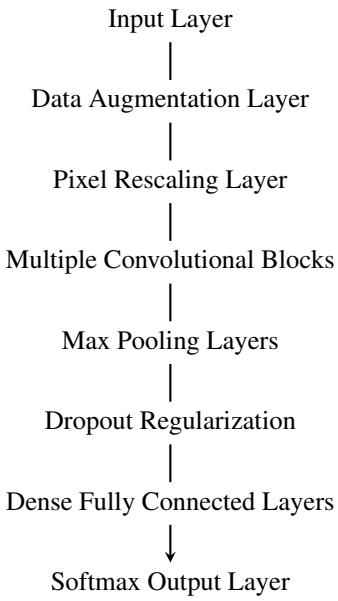
To improve generalization and reduce overfitting, each of the images were augmented to maintain meaningful information while reducing information that would be specific to each testing image and not generalizable to unseen data. Techniques included random horizontal flipping, image rotation, zooming, and contrast adjustments. This helps to increase the scope of the training set by simulating unobserved variations.

3 Methods

The primary tool utilized for constructing our prediction classifier was a Convolutional Neural Network (CNN) and was designed specifically for multi-class prediction. We will further define the structure of the model in the following sections.

3.1 Architecture

In the creation of the model, various core CNN components were incorporated including convolutional, max pooling, data augmentation, fully connected, softmax output, etc. layers. The model structure follows the outline listed below:



3.2 Input/Augmentation/Rescaling Layers

The first step of our model is to input and augment the data into a form that is more convenient both for storage, model construction, and training speed. The input layer is resized as 64x64x3 to represent a 64x64 pixel image with a RGB value at each pixel. Each image is effectively reformatted into 12,288 unique data points. This is important because it standardizes different satellite image sizes and resolutions into a consistent format that can be fed into the model.

After the data is input into the model, the data is augmented to prevent overfitting. Overfitting occurs when the model effectively memorizes the training data. Each image can be thought of as having two layers of information: class related patterns and image specific patters. The goal of data augmentation is to erase the image specific information such that the model is more appropriate for classifying unseen data.

For each image, we will perform four commonly used image transformations: random horizontal flip (0.50 prob. of flip), random rotation ($\pm$ 8%), random zoom ($\pm 10\%$), and random contrast adjustment ($\pm 10\%$). One should note that, specifically for the random rotation step, some pixels have to be interpolated as the rotated grid of observed pixels does not exactly match the 64 cubic pixel input.

The final step in the data preprocessing is to rescale the RGB value for each cell within the pixel matrix from the original 0-255 series of observations to a standardized 0-1 gradient in order to help stabilize the model and improve convergence [1].

3.3 Convolutional Layers

The convolutional layers are the core section of the model and where the majority of the model strength is derived from. These layers are used to identify important spatial patterns within each dataset class. These may include details such as vegetation texture, road/river structures, urban layouts, and agriculture. The model was constructed using four convolutional blocks with increasing kernel depths. This allows the model to learn general patterns at first before learning progressively more complex patterns. Kernel counts increase from 32-64-128-256 across the four batches. Each convolutional block contains two 2-D convolutional layers followed by batch normalization before a final max pooling layer at the end of each block.

3.3.1 Convolutional Operations

The convolution operation is defined as:

$$s(t) = (x * w)(t) = \int x(a)w(t-a)da \tag{1}$$

where x is the input signal/function, w is the kernel response/weight function, and a is the time/pixel shift. Since the time/pixel is generally treated as a discrete value, either some time or pixel index, $s(t)$ can be written in discrete summation form as:

$$s(t) = \sum_{a=-\infty}^{\infty} x(t)w(t-a) \tag{2}$$

Each convolutional layer applies adjustable kernels across the input surface map. For a convolutional kernel (K) of size 3×3, the convolutional operation at spatial position (i, j) may be written as:

$$(I * K)(i,j) = \sum_{m=-1}^{1} \sum_{n=-1}^{1} I(i-m, j-n)K(m,n) \tag{3}$$

where I represent the input surface map, K represents the convolutional kernel and (i, j) represents the spatial location of the output [2]. Each kernel computes a weighted sum of neighboring pixels, allowing the CNN to detect structures such as edges, textures, and boundaries.

Each convolutional layer is completed with the use of a Rectified Linear Unit (ReLU) activation function, defined as:

$$f(x) = \max(0, x) \tag{4}$$

in order to introduce non-linearity to the model. Using the ReLU activation function helps to prevent signal decay during backpropagation across multiple convolutional layers [3].

3.3.2 Batch Normalization Layer

The batch normalization layer is applied to stabilize feature distributions by using their sample statistics to compute outputs. For a mini-batch $B = (x_1, x_2, \ldots, x_m)$, batch normalization computes:

$$Mean: \mu_B \leftarrow \frac{1}{m} \sum_{i=1}^{m} x_i \tag{5}$$

$$Var : \sigma_B^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \tag{6}$$

The batch is then normalized via:

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \tag{7}$$

Batch normalization helps to aid in model convergence and promote numerical stability [4].

3.3.3 Max Pooling Layer

After each block of convolution/normalization layers, a max pooling layer was utilized using the default 2×2 max pooling operation defined as:

$$y = \max_{x \in R}(x) \tag{8}$$

effectively forcing the block to select the strongest feature detection and make the model more robust to small shifts in the image.

3.3.4 Convolutional Block Structure

Each convolutional block is constructed of two sets convolutions/batch normalizations, followed by a max pooling layer. In theory, we have created a model that reduces the input feature map progressively until it reaches its final output as a set of probabilities for each of our ten land cover classes. The progressive spatial dimension of the model is shown in Table 3.

Stage	Output Dimensions	Activation Values
Input	$64 \times 64 \times 3$	12,288
Block 1	$32 \times 32 \times 32$	32,768
Block 2	$16 \times 16 \times 64$	16,384
Block 3	$8 \times 8 \times 128$	8,192
Block 4	$4 \times 4 \times 256$	4,096

Table 3: Dimensionality reduction

Conceptually, one could imagine that each step examines increasingly more complex patterns, such that the blocks may identify $Pixels \rightarrow Textures \rightarrow SpatialStructures \rightarrow GeographicPatterns \rightarrow LandCoverClass$ respectively. As is typical due to the "black box" nature of Convolutional Neural Networks, we are unable to identify for certain what patterns each layer is detecting, but this is a reasonable conceptualization of the levels of complexity that the pixel information of each satellite image may contain.

3.4 Post-Convolutional Layers

3.4.1 Global Pooling Layer

After running the convolutional layers, the model flattens the $4 \times 4 \times 256$ convolutional output via a Global Pooling operation that reduces the information into a vector y of size 256. Each vector entry is calculated as:

$$y_k = \frac{1}{H \times W}\sum_{i=1}^{H}\sum_{j=1}^{W} x_{ijk} \tag{9}$$

where H, W are the spatial dimensions and x_{ijk} is the activation at spatial position (i, j) in feature map k. Essentially, we convert each feature map into one singular feature value.

3.4.2 Dense Layer

The dense layer connects each neuron to each input feature. Mathematically it performs the operation:

$$z = Wx + b \tag{10}$$

where x is the input feature vector, W is the weight matrix, b is the bias vector, and z is the vector of output activations.

3.4.3 Dropout Layers

Two dropout layers were used to randomly remove neurons, one after the global pooling layer (35% dropout rate) and another after the dropout layer (25% dropout rate). Dropout layers were incorporated to reduce overfitting by transforming each neuron output into:

$$\tilde{h}_i = h_i \cdot m_i \tag{11}$$

$$m_i \overset{iid}{\sim} Bernoulli(1 - p)$$

where p is the dropout probability for the given dropout layer.

3.4.4 Output Layer

The final layer of the model is the output layer that outputs a vector of size 10 via a softmax activation function defined mathematically as:

$$\Pr(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}} \tag{12}$$

that translates logits produced by the dense layer into probabilities.

4 Results

After defining the model, it was trained on the stratified training set with batch sizes of 32 to increase the rate of model fitting and avoid excessive numerical adjustments to the weights. Both the model loss and prediction accuracy were recorded for both the training and validation data at each of the 25 epochs during model fitting.

4.1 Model Loss

The form of model loss that was utilized while training the model was categorical cross-entropy due to the categorical output possibilities associated with land class data. Shannon (1948) establishes categorical cross-entropy mathematically as:

$$-\sum_{i=1}^{10} y_i \log(\hat{p_i}) \tag{13}$$

where y_i is the true probability distribution for the observation (1 for true class, 0 for others) and $\hat{p_i}$ is the probability distribution derived via the models softmax activation function from the computed logits. Average model loss at each epoch is shown below.

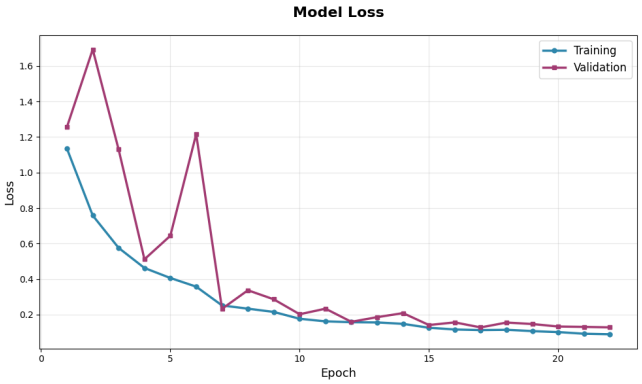


Figure 1: Model loss at each epoch via categorical cross-entropy.

We can see that the model learns from the training data fairly quickly and consistently, but struggles to generalize this learning to the validation set until around epoch 7-10 at which the model converges enough to stabilize the difference between the training and validation loss.

4.2 Prediction Accuracy

In addition to recording the model loss at each epoch, prediction accuracy on both data sets was also recorded. Since the output is a vector of ten class probabilities, we can extend the singular prediction accuracy out to any arbitrary subset of predictions. For our purposes, model fitting also recorded top 3 accuracy to record the percentage of instances where the actual class was among the top 3 predicted probabilities from the model response.

We can see visually, like in the loss function plot, the models accuracy demonstrated the difficulty of the model to generalize predictive ability to unseen data until around epoch 7-10. As the model progresses past epoch 15, the predictive power difference between the training and validation sets becomes nearly negligible.

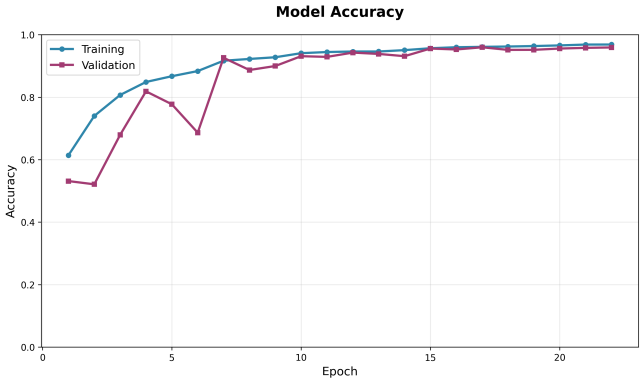


Figure 2: Model accuracy at each epoch.

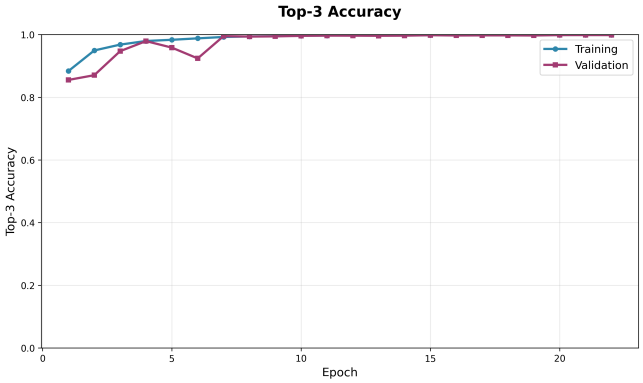


Figure 3: Top 3 model accuracy at each epoch.

4.3 Performance Metrics

We initially examined overall prediction metrics, but we can also constrict these metrics to each individual class in order to examine which classes the model identifies very well and which models it struggles to classify. We will define the following three metrics: precision, recall, and F_1 as:

$$Precision = \frac{TP}{TP + FP} \tag{14}$$

$$Recall = \frac{TP}{TP + FN} \tag{15}$$

$$F_1 = \frac{2(Precision)(Recall)}{Precision + Recall} \tag{16}$$

We can intuitively think of these metrics as follows: precision is how often the model is correct when predicting a given class, recall is how many actual instances of the class were predicted, and F_1 is a singular score incorporating both precision and recall. Our primary metric of interest is the F_1 score as it demonstrates the models balance between over predicting and under predicting a given class.

4.4 Addressing Misclassifications

Our model appears be best at identifying residential, industrial and water related land classes. One may logically deduce

Table 4: Model Performance Metrics by Class

Class	Precision	Recall	F_1 Score
Overall	96.08%	95.97%	95.97%
A. Crop	98.03%	91.33%	94.56%
Forest	91.73%	99.83%	95.61%
H. Vegetation	96.65%	91.50%	94.01%
Highway	97.96%	96.00%	96.97%
Industrial	98.00%	98.20%	98.10%
Pasture	96.63%	93.25%	94.91%
P. Crop	89.63%	96.80%	93.08%
Residential	96.61%	99.67%	98.11%
River	96.63%	97.40%	97.01%
Sea/Lake	98.97%	95.67%	97.29%

that this increase in accuracy is because of the more profound distinctions between building, waterway, and road edges. Differentiating between different vegetation covers may be more difficult as the model is likely to rely much more heavily on less distinct differences such as texture and color patterns that may vary depending on time of year and geographic location. We can examine the models confusion matrix to further understand where the model is having difficulty differentiating land classes.

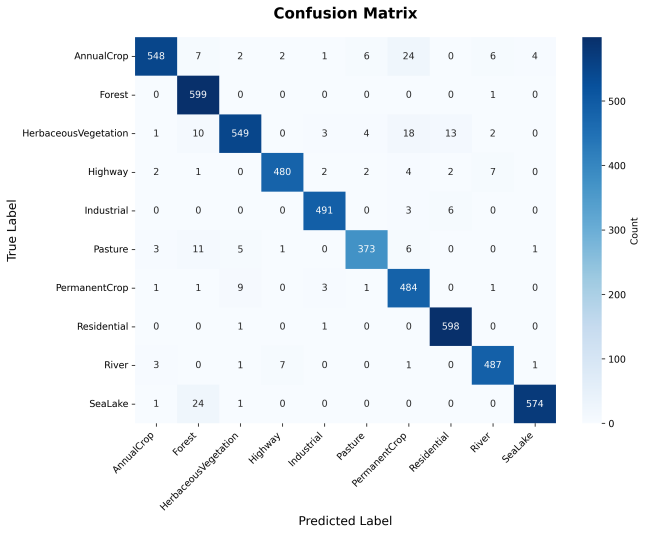


Figure 4: Model confusion matrix

By examining the confusion matrix, we can see that the model most commonly mistook annual crops for permanent crops (24), sea/lake for forest (24), herbaceous vegetation for permanent crops(18), herbaceous vegetation for residential (13), and pasture for forest (11). In Figures 5. and 6., we examine instances of misclassification for the two most common misclassifications.

While the complexity of the Convolutional Neural Network makes it impossible to deduce exactly what is causing the model to trip up, we can make some logical deductions by examining the nature of the problematic images. For example,

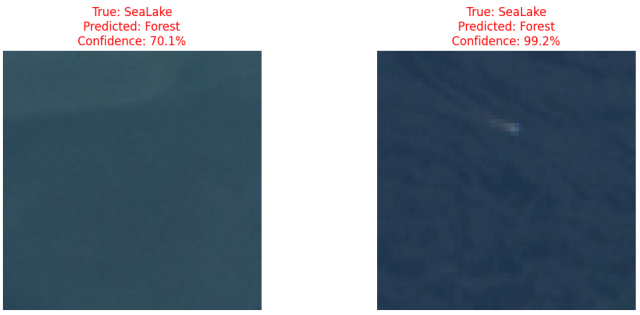


Figure 5: Misclassification examples (Sea/Lake for Forest)

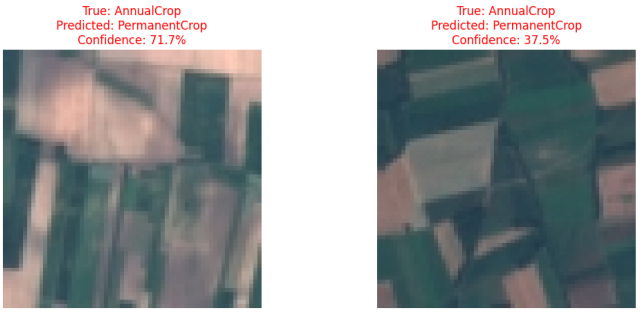


Figure 6: Misclassification examples (Annual Crop for Permanent Crop)

images of waterbodies found in the Sea/Lake class are likely to be solid, dark colored gradients, very similar to images from the forest class. While the texture of the forest canopy may help to distinguish the two classes, features like waves, sandbars, or cloud shadows may ”trick” the model. Our secondary visual example of misclassification in Figure 6. may be even more intuitive of an issue. Land for annual crops (corn, wheat, soybeans, etc.) and permanent crops (vineyards, orchards, etc.) are both fundamentally plotted agriculture land. Especially during certain times of the year, the visual differences that separate the two types may be reduced and therefore the model may rely more heavily on shared characteristics such as plot separation or structured vegetation. While we will stop at the examination of these two categories of misclassification, one could likely extend this intuitive thinking to all of the significant misclassification states.

At the very least, we should note the most significant types of model error in order to determine if it will be a significant issue for future applications. For example, if one were to use the model solely for identification of man-made structures in forested areas, the model would likely be sufficient for our purposes. However, if we wanted a model that specifically separated areas of annual and permanent crop coverage, our model may be too unreliable to base important decisions. In this case, and if the scope of our application was restricted solely to areas predetermined to be agricultural, we may want to retrain our model solely on satellite data from different agricultural areas so that it may capture more nuanced differences that we are comfortable fitting the model to if generalization is not a major concern.

5 Discussion

Upon analyzing the performance metrics, we can see that Convolutional Neural Networks appear to be a strong candidate for autonomous classification of land cover.

5.1 Model Strengths

In many ways our model accomplishes the goals we set out to accomplish. Specifically, we see strong overall classification strength for our model. The model performs best in identifying land cover that includes buildings or water features, based on analysis of F_1 metrics. In addition, we are confident in our models generalization ability as the accuracy and loss metrics nearly converged for the training and validation sets at later epochs. We may conclude based on our findings that the model is sufficiently strong for confidently separating general groupings such as farmland, cities, waterways, and untouched nature. Overall accuracy increases from 95.98% to 99.76% when we count a Top 3 prediction as a correct prediction, showing strong accuracy among the more general land cover groupings. If we specify more general groupings for the land cover classes (agricultural cover, natural cover, city cover, and waterway cover), overall accuracy decreases slightly to 97.22% but still higher than the more specific class case.

5.2 Limitations

The primary limitation of of the model is the lack of representation of different land types in the possible outcome classes. While the ten predefined classes are likely sufficient for a good number of applications or within the scope of specific geographic regions, there are certain use cases where the model will be insufficient. The model fails to account for a number of geographical regions, such as mountainous, snow covered, or desert. With no way to alternatively classify an image, the model will attempt to fit it as best as it can into one of the ten predetermined classes, potentially with a conflated level of confidence. To demonstrate this, we will manually extract an image from a glacial formation in a mountainous region of Alaska. Our model does not have any reasonable output choice for this satellite image, so our model output will almost certainly be nonsensical.

Shown in Figure 7. and Table 5, our model classifies this region as industrial with nearly 76% confidence, though there are clearly no man made structures present, much less a fully developed industrial district. However, according to our model, we would carry on under the assumption that this area would be an industrial region. If this misclassification is non-consequential or if we are certain our input pertains only to the ten predetermined classes, this may not be of much concern. However, if this is of concern, we may want to examine the use of another model or consider some mitigation method such as some form of presecreening or the addition of an "unidentified" class in the model fitting.

In addition to limited scope of the model, there are also classification errors within the bounds of the predictable classes.

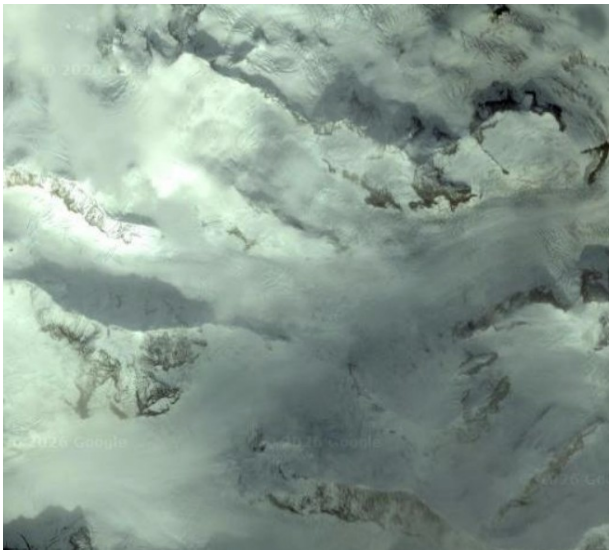


Figure 7: Glacial/mountainous region

Class	Probability
Industrial	75.96%
Herbaceous Vegetation	8.86%
River	7.20%
Permanent Crop	4.01%
Sea/Lake	3.16%

Table 5: Region classification

Overall classification strength is strong when utilizing the CNN model, but it is not perfect, especially between similar land cover groups. The accuracy is likely sufficient for many purposes, but it should still be noted, especially if the primary goal of the models utilization is in identifying more troublesome classes such as differentiating between crop land types or identifying herbaceous vegetation.

Finally, while in some cases a more generalized classification may be desired, it is also likely that we would desire a more specific classification, such as differentiating between crop types in agricultural plots. In a narrower project scope, refitting the model on a smaller subset unique to agricultural plots is likely more desirable. This would require obtaining and manually reclassifying a new set of data. In short, the model accuracy is largely sufficient for satellite classification, but research objectives greatly influence the applicability of a given model.

6 Conclusion

For the application of satellite imagery classification, it appears that Convolutional Neural Networks are a strong approach to utilize. We noticed a strong overall accuracy of approximately 96% when using such a model. Precision and recall across individual classes also demonstrated strong generalization for prediction outcomes, with no singular class witnessing overly-concerning drops in prediction metrics. In addition, model parameters seem to lead to sufficient model con-

vergence upon examining the loss and accuracy plots across the 25 model fitting epochs. Overall, we present a strong choice of model for classification purposes with specific case uses dictating adjustments which are allowed for by slight changes in feature construction and model fitting.

References

[1] Yang S, Xiao W, Zhang M, Guo S, Zhao J, Shen F. Image data augmentation for deep learning: A survey. *arXiv preprint arXiv:2204.08610*. 2022.

[2] Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (pp. 315–323). JMLR Workshop and Conference Proceedings.

[3] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

[4] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning* (pp. 448–456). PMLR.

[5] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

[6] Helber P, Bischke B, Dengel A, Borth D. EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*. 2019;12(7):2217–2226.